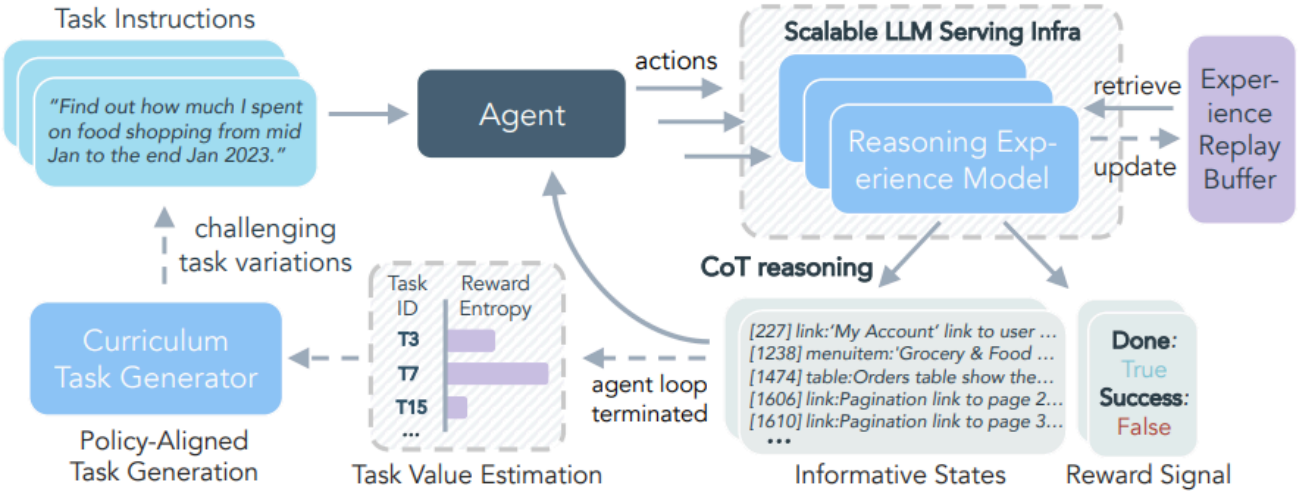
Scaling Agent Learning via Experience Synthesis

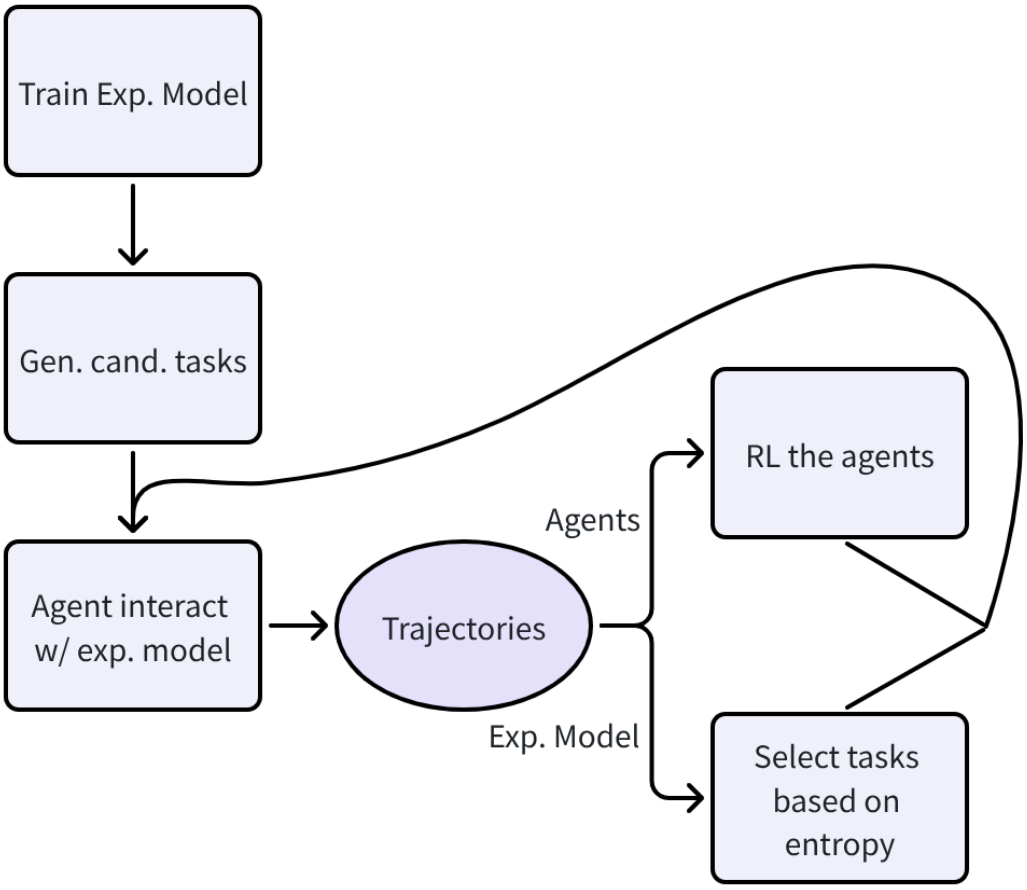
> #Agents #RL #Curriculum-based #SFT



- What questions does this project solve?
  - Traditionally, to train an agent, we render a real environment (*e.g., a browser page*). This is heavy and token costly (*e.g., a HTML based page has too many useless elements*).
  - To solve this problem, this paper proposes a way to use an **experience model** to simulate a "real" interaction environment with a **text-based** form.
- What can this framework do?
  - Use an experience model M_{exp} as the **textual** "environment" that the agent interacts with, instead of using a real environment where the process can be costly.
  - Finetuning of M_{exp}: Given a set of trajectories $\{(s_t, a_t, s_{t+1}, r_{t+1})\}$, use a model to generate reasoning of "why doing a_t in state $s_t$will lead to state $s_{t+1}$and gain reward r_{t+1}". Use **reasoning** and the **next-state** to run SFT to train the experience model to be good at simulating an environment.
  - Curriculum-based Task Generation: the **same** model $M_{exp}$is also responsible for generating new tasks. We generate $m$tasks as candidates. Let the agent system run each of the $m$candidates n times. We calculate entropy：

    $$\mathcal{V}_\tau = \frac{1}{n}\sum_{i=1}^{n}(r^i - \bar{r})^2, \quad \text{where } \bar{r} = \frac{1}{n}\sum_{i=1}^{n} r^i$$

    If entropy is non-zero, meaning at some time the agents can solve the task and others cannot, this task is valuable to use (neither too easy nor unsolvable).
  - RAG: the pipeline will maintain (update) a RAG, initially with some real traces added by researchers. When interacting with the agent, the experience model will first have a check in RAG to avoid hallucination.
  - Usage:
    - Standard:



    - Sim-to-Real Policy Transfer: After the *Standard Cycle*, enter the **real environment** to do some minor training. Result: 10% real data -> 40% better than purely real environment.
- My thoughts
    - Pre-filter: To filter out candidate tasks with proper difficulties, we need to run the cycle first. If there are many too-easy or unsolvable questions, will the trajectories affect the RL? Also this has high latency. Is there a way to pre-filter the candidates (like using another frozen powerful LLM)?
- Work Cited
    - Paper
    - Chat History